Review package: operation-scoped cancellation (B1–B7)

> **Scope note.** This packet reviews the **operation-scoped-cancellation** work. It supersedes the earlier "browser lifetime + stop rule S4" packet (candidate `9a1b931`, base `b676c2d`), which is a separate change preserved in git history. Nothing here touches BM25/aboutness/discovery/counters/scoring/operating-settings, `BATCH_STOP_RULE_FAILED_JOBS` stays unset, and there is no quarantine-recovery override. **This is a local feature-branch review candidate only: no merge, no deploy, no paid/Railway runs, no live web/LLM calls.**

Branch	`fix/operation-scoped-cancellation`
Base (production)	**`origin/main = 8c03b07449701a7fb83dc062abbfe6726cf7c208`** — untouched; nothing pushed to it
Release candidate	**`6e09ced4bf42d036c257cb35772335600215ff30`**
Lineage	`f764c95` (B1 recovery scope + B6 killable pdf-parse subprocess) → `b317833` (B2/B3/B7 production-boundary tests + B7 test seam) → `0d05556` (recovery window = real remaining fetch-phase budget) → `782f99d` (bounded in-flight browser interruption, PDF path) → `e4894ae` (obsolete-work settlement for coalesced callers) → `45cc661` (real-IPC proc-admission withdrawal test + explicit unattempted-URL outcomes) → `658a17c` (deadline threading into browser nav timeouts via `remainingMs`) → **`6e09ced`** (bounded in-flight interruption wired into the `fetchWithBrowser` HTML path + abort-aware settle waits)
Gates	No new env switch. The cancellation machinery is active on the isolated-child path that production already runs (`WORKER_ISOLATION_ENABLED=true`). `BATCH_STOP_RULE_FAILED_JOBS` is **not** part of this change and stays unset.
Delivery constraints honoured	Feature branch only; **no push to origin/main**; no merge; no deploy; no paid/Railway runs; no real web/LLM calls.

---

1. What changed (`origin/main..6e09ced`)

Code:

- **`server/lib/pipeline.ts`** —
  - `runCompanyPdfRecovery` arms **one** `OperationScope` per company

PDF recovery, caps its deadline by `min(PDF_RECOVERY_MAX_BUDGET_MS, remainingMs)`, propagates that scope to **every** fetch call (`fetchFn(origin, slice, { outcomes, scope })`), stops dispatching once the scope aborts (leaving the untried tail retryable), and caps total attempts across origins.
    - **(0d05556)** `remainingFetchBudgetMs(fetchPhaseStart)` returns the real remaining fetch-phase budget (configured budget minus elapsed, clamped to `RECOVERY_WINDOW_FLOOR_MS`), and the recovery call site now feeds **that** as `remainingMs` — so the production recovery window is the per-company remaining budget, not the raw 600000 ms constant. The constant itself is unchanged.
    - **(45cc661)** `markUnattemptedCancelled(companyId, companyName)` + const `NOT_ATTEMPTED_CANCELLED_REASON = "not_attempted_cancelled"`: on a cancel break in the fetch loop, every still-`pending` accepted document is recorded fetch-dead with that reason, so cancellation no longer leaves URLs silently `pending`. Wired into the top-of-loop cancel break (non-fatal, wrapped in try/catch).
- **`server/lib/processor.ts`** —
    - **B2:** `acquireProcSlot` queued waiter is cancellable — on abort it withdraws from the local broker queue and consumes no slot; grant/abort races hand the slot straight back. In the isolated-child path it withdraws across IPC via `remoteAcquire("proc", signal, () => new OperationAbortedError(…, "proc_admission"))`.
    - **B3:** `runProcessCollect` registers an abort listener that SIGKILLs a hung extraction child at the abort instant.
    - **B6:** pdf-parse runs in a **killable subprocess** (`extractViaPdfParseSubprocess` → `pdf-parse-worker.cjs` via `runProcessCollect`), not in-process — a genuine execution boundary.
    - **B7:** `processDocument` coalesces concurrent same-URL callers to one fetch **only while the in-flight scope is live**; a caller arriving after that scope aborts starts its own attempt (liveness guard). **(e4894ae)** a caller that *coalesced onto* an in-flight op and whose **own** scope then aborts self-settles via `Promise.race([existing.promise, callerScope.whenAborted("coalesced_wait")])` instead of hanging on the obsolete shared op.
    - **(782f99d)** `installBoundedPageInterrupt` + `BROWSER_ABORT_GRACE_MS` (500 ms): on scope abort, an in-flight browser op is driven to settle within the grace bound + a gate round-trip instead of running to its own timeout. In `782f99d` this was wired only on the PDF path.
    - **(this candidate)** the bounded interrupt is now **also wired into the `fetchWithBrowser` HTML path and into `fetchIssuerPdfsWithPrimedSession`** (`clearInterrupt = installBoundedPageInterrupt(page, scope)` at lines ~1506/2021/2298), closing the §6.1 “HTML path not wired” gap from the prior candidate. Two supporting HTML-path changes: the two post-navigation settle waits become `abortableDelay(2000, scope)` (resolve early on abort instead of always burning 2 s), and the catch block converts the puppeteer “Target closed” / detached-frame rejection produced by the forced page-close **after** an abort into a retryable `OperationAbortedError(…, "browser_html_interrupted")`, so an interrupted HTML fetch surfaces an explicit abort instead of a silent `""` (dead URL). Deadline threading into the per-navigation timeouts (`gotoTimeout` clamped to `remainingMs`)

landed in `658a17c`.

Tests added / present (all under `server/lib/`): `company-pdf-recovery-boundary.test.ts` (B1, incl. B1-7 recovery window), `cancellable-proc-admission-boundary.test.ts` (B2 in-process), `isolated-proc-admission-boundary.test.ts` (B2 across real fork+IPC) + fixture `__fixtures__/proc-admission-child.ts`, `stalled-extraction-boundary.test.ts` (B3), `pdf-parse-subprocess-equivalence.test.ts` (B6), `inflight-coalescing-boundary.test.ts` (B7, incl. B7-3 obsolete-work settlement), `browser-inflight-interrupt.test.ts` (bounded in-flight page interrupt, real Chromium — **now 2 subtests: PDF path + the new HTML `fetchWithBrowser` path**), `unattempted-cancelled-boundary.test.ts` (explicit unattempted-URL outcomes, real Postgres), `operation-scoped-cancellation.test.ts` (scope primitives A/B/C) and the pre-existing `browser-close-order.test.ts` (real-Chromium close/reap ordering).

---

2. Coverage matrix

Full requirement→test→assertion map, with the realism level of each case and the honest gaps, is in **COVERAGE_MATRIX.md**. Headline: **11 test files, all green on `6e09ced`** — 34 `node:test` subtests across 10 files + `scoring-cancellation.unit` (60 passed) = **94 total checks, 0 failures**. Realism spans a real-killed child process (B3), a real fork+IPC admission withdrawal (B2b), a real Chromium child close/reap on four exit paths, a real Chromium page bounded-interrupt on **both** the PDF and HTML browser paths, real Postgres terminal-state assertions, and the real production broker/scope functions in-process for the rest.

---

3. Mutation-fail demo per guard (all run this session, applied → observed → reverted clean)

Each guard is shown to be load-bearing: remove it and the named assertion fails. Every mutation was reverted and the tree re-verified clean (`grep -rn MUTATION_MARKER server/` → none; `git diff --quiet HEAD -- server/lib/processor.ts server/lib/pipeline.ts` → clean).

Gap	Mutation	Result	
B1 scope propagation	`fetchFn(origin, slice, { outcomes, scope })` → `{ outcomes }`	**B1-1 fails**	As wa bo
B1-7 recovery window	`remainingFetchBudgetMs` body → `return budgetMs;` (`MUTATION_MARKER_B2`)	**B1-7 fails**	6 n el
B2 queued-waiter abort	remove `sig.addEventListener("abort", onAbort…)`	**B2-1 fails**	'h 2/ le
B2b IPC withdrawal error type	`() => new OperationAbortedError(…, "proc_admission")` → `() => new Error("MUTATION_MARKER_B5 …")`	**isolated-proc-admission fails fast**	cl m fa

B3 stalled-child kill	disable abort registration	**B3 fails**	pa af (e ra
B7 coalescing	`if (false && existing …)`	**B7-1 fails**	2
B7 liveness	ignore `scope.aborted()`	**B7-2 fails**	1 co er
B7-3 coalesced self-settle	`Promise.race([existing.promise, callerScope.whenAborted(…)])` → `return existing.promise;` (`MUTATION_MARKER_B4`)	**B7-3 fails**	2 co it:
B-browser (PDF) bounded interrupt	`installBoundedPageInterrupt` `onAbort` body → no-op (`MUTATION_MARKER_B3`)	**browser-inflight-interrupt PDF subtest fails**	pa 3(
B-browser (HTML) `fetchWithBrowser` wiring	HTML-path wiring neutralised only (`clearInterrupt = () => {}`) in `fetchWithBrowser`, PDF path left intact)	**HTML subtest fails, PDF subtest still passes**	H **m** BF vs ne in
B6(unattempted) terminal outcomes	pending filter `&& false` (`MUTATION_MARKER_B6`)	**B6-1 fails**	0 si

B7 needs three mutants because the coalesce / liveness / self-settlement clauses are independently load-bearing. The B2b *error-type* mutation is the load-bearing one for the IPC path; a weaker `cancelAcquire`-suppression mutation did **not** fail (child-exit cleanup masks it) and that masking is documented in COVERAGE_MATRIX §2b rather than hidden.

---

4. Gate results (`6e09ced`)

All 11 cancellation test files green (fixtures: local `ciq-int-pg` on 55432, `ciq-int-redis` on 56379; no production `DATABASE_URL`; `PDF_RECOVERY_MAX_BUDGET_MS=600000`):

```
operation-scoped-cancellation        pass 7  fail 0
company-pdf-recovery-boundary        pass 7  fail 0   (B1-7 added)
cancellable-proc-admission-boundary  pass 4  fail 0
isolated-proc-admission-boundary     pass 1  fail 0   (real
fork+IPC)
inflight-coalescing-boundary         pass 3  fail 0   (B7-3 added)
stalled-extraction-boundary          pass 2  fail 0
pdf-parse-subprocess-equivalence     pass 2  fail 0
browser-close-order (real Chromium)  pass 4  fail 0
browser-inflight-interrupt (real)    pass 2  fail 0   (PDF + HTML
paths)
unattempted-cancelled-boundary (PG)  pass 2  fail 0
scoring-cancellation.unit            60 passed, 0 failed   (custom
runner)
```

34 `node:test` subtests + 60 scoring-unit assertions = **94 total checks, 0 failures**. `browser-close-order` and `browser-inflight-interrupt` ran against a **real** Chromium (`PUPPETEER_EXECUTABLE_PATH`), not skipped; `isolated-proc-admission-boundary` spawned a **real** OS child over the production IPC channel.

---

5. Normalized tsc comparison

`npx tsc --noEmit -p tsconfig.json`, error sets path/line-normalized and de-duplicated, candidate vs base (base run in a worktree checked out at `origin/main` = `8c03b07` with the same `node_modules`). Normalization: `grep 'error TS' | sed -E 's/\(([0-9]+),([0-9]+)\)//' | sort -u` (unique error *signatures*, not raw line count).

	unique error signatures
base `8c03b07`	13
candidate `6e09ced`	12
new errors in candidate (`comm -13`)	**0**
errors removed by candidate (`comm -23`)	1 — `server/lib/processor.ts … TS7016: Could not find a declaration file for module 'pdf-parse'` (eliminated because B6 moved pdf-parse out of the in-process import)

The candidate introduces **zero** new type errors and removes one pre-existing `pdf-parse` declaration warning. All 12 remaining signatures are pre-existing vitest/`any` noise identical to base. (This counts unique error *signatures*; the earlier packet's "62/61" figure counted raw error *lines* — the invariant is the same either way: **0 new errors**.)

---

6. Remaining unbounded / unproven paths (honest boundary)

1. **(RESOLVED in this candidate)** *Active in-page CDP interruption was bounded on one path only.* In the prior candidate `installBoundedPageInterrupt` was wired only into `fetchPdfViaBrowser`, leaving the `fetchWithBrowser` HTML path uninterruptible. It is **now wired into both browser paths and the primed-session path** (lines ~1506/2021/2298), proven with a real Chromium page by the new HTML subtest 5b.2 and its path-isolated mutation (wired → 8108 ms when neutralised, while the PDF subtest still passes). A fetch / `page.goto` / `page.evaluate` / WAF challenge already executing on the HTML path when the scope aborts is now driven to settle within `BROWSER_ABORT_GRACE_MS` + a gate round-trip and surfaces a retryable `OperationAbortedError(…, "browser_html_interrupted")`, instead of running to the browser timeout or returning a silent `""`.
2. **Deadline is now threaded into the browser fetch at the navigation + settle layers; a deep per-socket timeout is still not asserted.** Since `658a17c` the per-navigation timeouts are clamped to `remainingMs` (`gotoTimeout = base => scope ? max(1,

`min(base, scope.remainingMs()) : base`, lines ~1496/2009), and this candidate makes the two post-navigation settle waits abort-aware via `abortableDelay`. So on both browser paths the effective ceiling on an in-flight fetch is now the scope deadline (via the clamped nav timeout) or the grace bound (via the interrupt), **not** the pipeline-deadline → SIGKILL fallback. What is still **not** independently asserted by a test is a per-request TCP/socket read timeout equal to `remainingMs` inside the lowest-level puppeteer/CDP transport; the abort path tears the page down within the grace bound, which supersedes that socket in practice, but no test pins the socket layer itself. This is the honest residual of the former "bounded, not threaded" gap.

3. **(RESOLVED in the `0d05556` candidate)** The production recovery window previously degenerated to the `PDF_RECOVERY_MAX_BUDGET_MS` (600000) constant because the call site omitted `remainingMs`. It now feeds `remainingFetchBudgetMs(fetchPhaseStart)` — the configured budget minus elapsed, clamped to a floor — so the per-company remaining budget is honoured in production, not just in the test (B1-7 + mutation). The constant is unchanged; only the derivation changed.
4. **Specific production stalls unobserved.** The Idemitsu ~18-minute single step and the AXA analysis-prep ~7m26s stall (`TIMEOUT_INVESTIGATION.md`) are not reproduced here and are **not** claimed fixed by this patch.

---

7. Claims kept narrow

- **Candidate only.** `6e09ced` is a review candidate on a feature branch. Nothing was merged, deployed, or run on Railway/paid providers. `origin/main` is untouched at `8c03b07` (verified: `git rev-parse origin/main` = `8c03b07…`; local `main` = `8c03b07…`).
- **No full scoring fixture.** Per the reviewer's clarification, no end-to-end scoring run is included; the behavioural tests drive the real caller-to-recovery boundary and the real admission/coalescing/subprocess/IPC/Postgres functions directly.
- **Mutations are session-local and reverted.** Every §3 mutation was applied, observed to fail the named assertion, and reverted in this session; `grep -rn MUTATION_MARKER server/` is empty and `git diff --quiet HEAD -- server/lib/processor.ts server/lib/pipeline.ts` is clean.
- **What is proven** is the requirement→assertion map in COVERAGE_MATRIX.md plus the §3 mutation demos. **What is not proven** is in §6 and COVERAGE_MATRIX §6 — read the green suite as exactly that boundary, no wider.

---

8. Reviewer's 6-point clarification — status

1. **Behavioural recovery boundary (not callee+grep).** Done — `runCompanyPdfRecovery` is real production code called in the production pipeline; B1-1..B1-7 assert scope construction, deadline capping, no-dispatch-after-expiry, retryable tail, healthy path, expiry-through-the-real-admission-boundary, **and** that the production window is the real remaining budget (B1-7). Mutations: scope-drop → B1-1 fail; `return budgetMs` → B1-7 fail (§3).

2. **Chromium realism + coverage matrix.** Done — COVERAGE_MATRIX.md states per case whether it is real-Chromium-child (R), real-Chromium-page (R(page)), real-subprocess (S), real fork+IPC (I), real-Postgres (D), or broker/unit (U). The bounded in-flight interrupt is now proven on **both** the PDF and HTML (`fetchWithBrowser`) browser paths (§5b, 5b.1/5b.2); the one remaining active-interruption residual — a deep per-socket read timeout not asserted by a test — is listed explicitly (§6.2).
3. **Fixture infra.** Done — `ciq-int-pg` (55432) / `ciq-int-redis` (56379), no production `DATABASE_URL`.
4. **B6 killable execution boundary.** Done — subprocess, not in-process size cap (§1); `pdf-parse-subprocess-equivalence.test.ts` green. Admission withdrawal additionally proven across a real fork+IPC boundary (`isolated-proc-admission-boundary.test.ts`).
5. **Recovery budget value retained, semantics documented.** `PDF_RECOVERY_MAX_BUDGET_MS=600000` retained; the production window now derives from the real remaining fetch-phase budget (B1-7), resolving the earlier “degenerates to the constant” gap. Documented in §6.3 and tested.
6. **Delivery: feature branch, no origin/main, no deploy/paid runs.** Honoured (§ header + §7).